

Reto

Unidades de trabajo:

Modelos de Inteligencia Artificial

Programación de Inteligencia Artificial

Big Data Aplicado

Modelos de Lenguaje

El Reto

Contexto: Formas parte de un equipo de desarrollo en **DataBot Solutions**, una empresa de consultoría tecnológica que trabaja con clientes de diferentes sectores para aprovechar al máximo el valor de sus datos. En esta ocasión, tu equipo ha recibido el encargo de uno de los clientes más importantes.

El cliente enfrenta el desafío de administrar y extraer información de grandes volúmenes de datos que actualmente están dispersos en distintos medios y además algunos son de constante actualización y además algunos no disponibles en una API. Estos datos, de gran importancia para la toma de decisiones, cubren diversas áreas y, aunque son accesibles, la consulta y combinación de la información es lenta y compleja para los usuarios no especializados. La falta de una herramienta que permita realizar preguntas en lenguaje natural y recibir respuestas con algo más de inteligencia hace que el proceso de extracción de información sea una tarea tediosa, susceptible a errores y poco eficiente para el equipo.

DataBot Solutions se ha consolidado en el sector tecnológico como una empresa líder en innovación aplicada, especializándose en el desarrollo de soluciones inteligentes y personalizadas que mejoran la eficiencia de las organizaciones. Con el aumento de la competencia y la necesidad de una gestión eficaz de la información, la compañía se ha fijado tres áreas fundamentales para el desarrollo de este proyecto:

Requisitos Funcionales por parte del cliente

1. **Obtención Automatizada de Datos Vía Scraping:** El sistema debe contar con una función de scraping automatizado que permita extraer información de sitios web relevantes, noticias del sector y otras fuentes públicas de la industria. Esta información deberá ser procesada de forma continua para alimentar el sistema con datos actualizados y relevantes para el análisis.
2. **Gestión y Optimización del Almacenamiento de Datos:** Para garantizar una consulta eficiente, el sistema debe estructurar y almacenar los datos en un entorno optimizado para búsquedas rápidas. Esto implica diseñar y gestionar un núcleo de almacenamiento centralizado y distribuido, donde los datos se indexen y se almacenen de forma accesible y escalable.
3. **Desarrollo de un Chatbot Inteligente:** El chatbot deberá ser capaz de interpretar preguntas en lenguaje natural y ofrecer respuestas rápidas y coherentes, extrayendo la información de los datos

almacenados en el sistema. Deberá comprender y responder consultas que van desde preguntas simples hasta aquellas que requieren la comparación o el análisis de datos en tiempo real.

Para garantizar la flexibilidad y facilitar la integración con otras aplicaciones y sistemas, el proyecto debe incluir una API robusta y bien documentada. Esta API permitirá la interconexión de servicios.

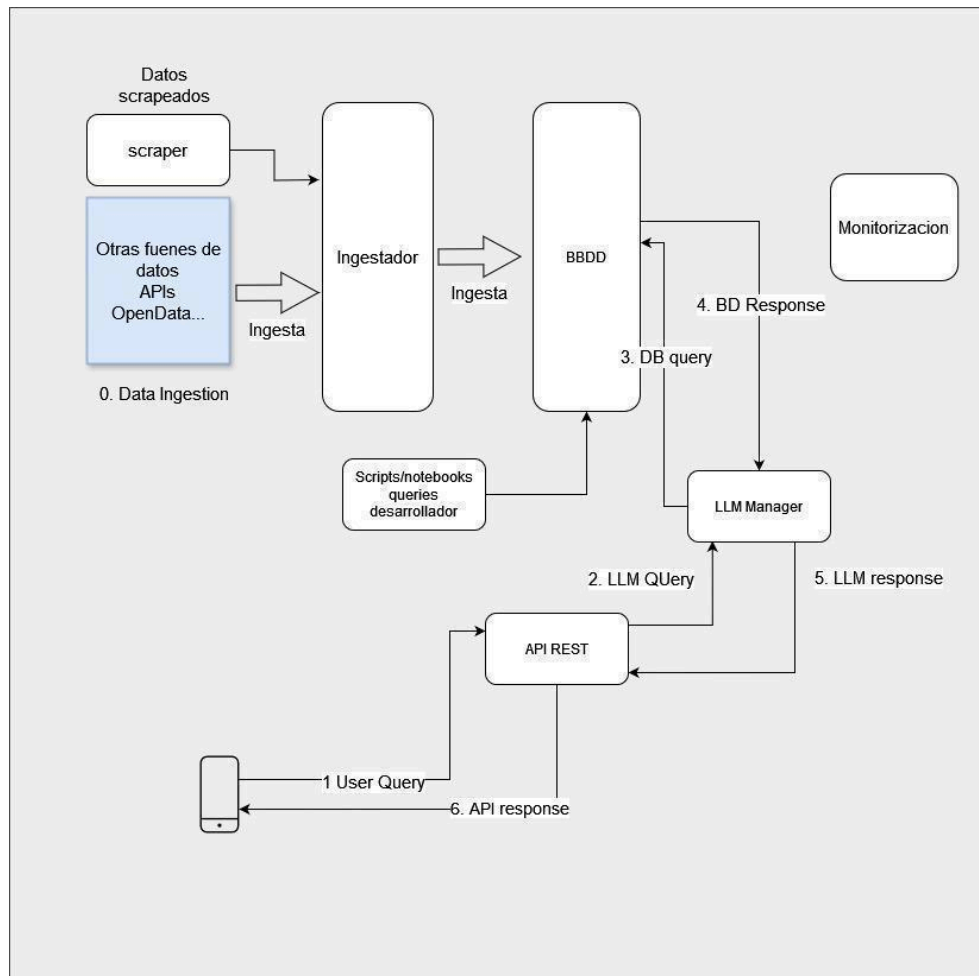
Requisitos Técnicos

1. **Indexación y Búsqueda Rápida con un Sistema Distribuido:** Para la gestión y almacenamiento de datos, el sistema debe utilizar una estructura de almacenamiento con capacidades de indexación avanzada y búsqueda rápida. El objetivo es crear índices optimizados que permitan acceder a grandes volúmenes de datos de forma eficiente, incluso en consultas complejas. El sistema debe también garantizar la alta disponibilidad de los datos mediante un clúster distribuido.
2. **Escalabilidad y Flexibilidad en la Integración de Fuentes de Datos:** El sistema debe ser fácilmente escalable, permitiendo la integración de nuevas fuentes de datos en el futuro, ya sea mediante scraping o API. Este diseño debe permitir que el asistente incorpore fuentes adicionales sin interrumpir el funcionamiento general.
3. **Optimización de Consultas y Respuestas en Tiempo Real:** La combinación de datos obtenidos automáticamente y los datos preexistentes debe estar organizada de manera que el chatbot pueda responder usando esos datos.
4. **Capacidad de Registro y Análisis del Historial de Consultas:** El sistema deberá almacenar un registro de consultas y respuestas para permitir la revisión de las interacciones, proporcionando insights sobre el uso del asistente y áreas de mejora para futuras versiones.

Con estos desafíos en mente, **DataBot Solutions** ha lanzado un reto a un grupo selecto de trabajadores: desarrollar un prototipo de chatbot que demuestre la capacidad de obtener información a través de scraping, almacenarla de manera eficiente en Elasticsearch y responder de forma precisa a las consultas de los usuarios. Además, el equipo deberá asegurar que el sistema sea escalable, seguro, y pueda soportar un gran volumen de datos de entrada.

La **dirección de DataBot Solutions y sus clientes** espera que al finalizar el reto, se presente un sistema de chatbot funcional que pueda ser probado en un entorno controlado, un conjunto de visualizaciones relevantes que reflejen el estado del sistema, así como evidencias del rendimiento del clúster y la infraestructura desplegada. El éxito en este reto marcaría el inicio de una implementación a mayor escala, demostrando cómo el poder de los datos y la automatización puede transformar la eficiencia y la capacidad de respuesta en las organizaciones.

Como pequeña ayuda se ofrece un diagrama aproximado (y sólo orientativo) de un sistema parecido que se usó en otro proyecto.



Objetivos / Resultados de Aprendizaje /ODS

Objetivos del reto:

1. Diseñar y Configurar un Sistema de Ingesta de Datos Escalable.
2. Implementar y Administrar un Clúster de Elasticsearch Distribuido.
3. Optimizar el Procesamiento y el Almacenamiento de Datos en BD (Elasticsearch).
4. Garantizar la Disponibilidad y Redundancia de los Datos.
5. Visualizar datos y monitorizar el estado del software desarrollado y servidores de operaciones.
6. Implementar un sistema de adquisición de datos no dependiente de API.
7. Implementar una API de acceso a los servicios propuestos y de interconexión con los servicios.
8. Desarrollar un ChatBot inteligente capaz de entender preguntas en lenguaje natural y responder de manera coherente, proporcionar respuestas rápidas y precisas en tiempo real, analizar y comparar datos para responder preguntas complejas, manejar preguntas simples y complejas de manera efectiva.

TRANSVERSALES

- Comunicación
 - Se evaluará mediante la presentación.
- Competencia digital
 - Se evaluará mediante el trabajo en Github en sus repositorios de trabajo.

OBJETIVOS DE DESARROLLO SOSTENIBLE



Resultados de Aprendizaje:

5071. Modelos de Inteligencia Artificial.

RA3. Relaciona el procesamiento de lenguaje natural con sus aplicaciones determinando su potencial e identificando sus limitaciones.

Criterios de evaluación:

- a) Se ha caracterizado el procesamiento de lenguaje natural.
- b) Se ha justificado el papel del lingüista en un proyecto de Inteligencia Artificial.
- c) Se ha determinado el potencial de las técnicas existentes de procesamiento de lenguaje, así como sus limitaciones.
- d) Se ha considerado en qué casos es factible aplicar estas técnicas en la resolución de un problema.
- e) Se ha evaluado el trabajo cooperativo entre lingüistas e informáticos en el campo del procesamiento de lenguaje natural.
- f) Se ha descrito la formación teórica que precisa el investigador en procesamiento de lenguaje natural.
- g) Se ha elaborado un sistema de procesamiento de lenguaje orientado a una tarea específica.

Contenidos: Procesamiento de lenguaje natural.

- Procesamiento de lenguaje natural: potencial y limitaciones.
- Aplicaciones del procesamiento de lenguaje natural.

RA6: Aplica principios legales y éticos al desarrollo de la Inteligencia Artificial integrándolos como parte del proceso.

Criterios de evaluación:

- a) Se han argumentado los posibles riesgos legales y éticos de la aplicación de Inteligencia Artificial.

- b) Se ha reconocido la necesidad de respetar la privacidad de los datos.
- c) Se ha decidido el cumplimiento estricto de la legalidad en su aplicación.
- d) Se ha integrado como parte del proceso la protección frente a previsibles errores y ataques (security by design).
- e) Se ha comprobado que se cumplen todas las normas legales y éticas en todas las áreas de la Inteligencia Artificial (privacy by design).
- f) Se han identificado y corregido los posibles sesgos de género en el desarrollo y aplicaciones de Inteligencia Artificial y Big Data.

Contenidos: Aplicación de principios legales y éticos de la Inteligencia Artificial.

- Deontología profesional en Inteligencia Artificial.
- Privacidad de datos.
- Protección frente a errores.
- Principios éticos.
- Sesgos de género en el desarrollo y aplicaciones de Inteligencia Artificial y Big Data.

5073. Programación de sistemas de inteligencia artificial

RA1: Caracteriza lenguajes de programación valorando su idoneidad en el desarrollo de Inteligencia Artificial.

- Principales características en un lenguaje de programación para Inteligencia Artificial. Bibliotecas. Rendimiento en ejecución. Herramientas. Soporte.
- Principales lenguajes de programación para Inteligencia Artificial: Python, R, Java, Javascript, NodeJS, JSON, entre otros.
- Programa informático. Etapas. Fundamentos del lenguaje de programación.

RA2: Desarrolla aplicaciones de Inteligencia artificial utilizando entornos de modelado.

- Módulos predefinidos
- a) Se han evaluado plataformas de Inteligencia Artificial.
- b) Se han caracterizado entornos de modelo de aplicaciones de Inteligencia Artificial.
- c) Se ha definido el modelo que se quiere implementar según el problema planteado.
- d) Se ha implementado la aplicación de Inteligencia Artificial.
- e) Se han evaluado los resultados obtenidos.

RA3: Evalúa las mejoras en los negocios integrando convergencia tecnológica.

- a) Se han identificado las ventajas que ofrece unificar procesos, servicios, herramientas, métodos y sectores.
- b) Se han identificado sistemas que facilitan la conexión tecnológica.
- c) Se han evaluado las características de dichos sistemas.
- d) Se ha evaluado cómo la convergencia tecnológica aporta seguridad en los negocios.
- e) Se ha evaluado la mejora en la capacidad de toma de decisiones estratégicas en un negocio conectado.

RA4: Evalúa modelos de automatización industrial y de negocio relacionándolos con los resultados esperados por las empresas.

- a) Se han identificado las nuevas estrategias corporativas y modelos de negocio en las empresas.
- b) Se han definido la relación entre empresas y clientes y su efecto en la forma en que las

empresas organizan y gestionan sus activos y recursos.

c) Se han evaluado modelos de automatización para los nuevos requerimientos industriales y de negocio.

d) Se ha evaluado la conveniencia de cada modelo para conseguir los resultados esperados por las empresas.

5075. Big Data Aplicado

RA1: Gestiona soluciones a problemas propuestos, utilizando sistemas de almacenamiento y herramientas asociadas al centro de datos.

Criterios de evaluación:

- Se ha caracterizado el proceso de diseño y construcción de soluciones en sistemas de almacenamiento de datos.
- Se han determinado los procedimientos y mecanismos para la ingesta de datos.
- Se ha determinado el formato de datos adecuado para el almacenamiento.
- Se han procesado los datos almacenados.
- Se han presentado los resultados y las soluciones al cliente final en una forma fácil de interpretar.

RA2. Gestiona sistemas de almacenamiento y el amplio ecosistema alrededor de ellos, facilitando el procesamiento de grandes cantidades de datos, sin fallos y de forma rápida.

- Se ha determinado la importancia de los sistemas de almacenamiento para depositar y procesar grandes cantidades de cualquier tipo de datos rápidamente.
- Se ha comprobado el poder de procesamiento de su modelo de computación distribuida.
- Se ha probado la tolerancia a fallos de los sistemas.
- Se ha determinado que se pueden almacenar tantos datos como se desee y decidir cómo utilizarlos más tarde.

RA3. Genera mecanismos de integridad de los datos, comprobando su mantenimiento en los sistemas de ficheros distribuidos y valorando la sobrecarga que conlleva en el tratamiento de los datos.

- Se ha valorado la importancia de la calidad de los datos en los sistemas de ficheros distribuidos.
- Se ha valorado que a mayor volumen de tratamiento de datos corresponde un mayor peligro relacionado con la integridad de los datos.
- Se ha reconocido que los sistemas de ficheros distribuidos implementan una suma de verificación para la comprobación de los contenidos de los archivos.
- Se ha reconocido el papel del servidor en los procesos previos a la suma de verificación.

RA5. Valida las técnicas de Big Data para transformar una gran cantidad de datos en información significativa, facilitando la toma de decisiones de negocios.

- Se han seleccionado gran cantidad de datos estructurados y no estructurados para reforzar la función de BI.
- Se ha realizado la limpieza y transformación de datos en base a los objetivos predeterminados.
- Se ha comprobado que el Big Data multiplica la relevancia y la utilidad del BI para el negocio.
- Se han conjugado dentro de un modelo de empresa datos de clientes, financieros de ventas, de productos, de marketing, de redes sociales, de la competencia, entre otros, para extraer un análisis valioso y efectivo para el negocio.

- Se ha evaluado e interpretado la información extraída de los datos y su influencia en el triunfo de diferentes negocios.
- Se ha simulado la implantación de un modelo de Inteligencia de Negocios (BI).

1. Tareas a realizar

Big Data Aplicado

1. Ingesta de datos:

- **Preparar la ingesta de datos** en logstash.
 - Crear archivos de configuración de logstash para subida de datos.
 - Utilización de distintos plugins según el tipo de datos, obligatorio buscar un uso para procesar con **grok**.
 - En el **output de logstash**, el índice del log de scrapping se tiene que actualizar con una variable dinámica que se actualice cada día.
(<https://www.elastic.co/guide/en/logstash/current/plugins-outputs-elasticsearch.html>).
 - Gestión utilizando pipelines para la subida de los distintos datos.
- **Automatizar la recopilación** de métricas operativas de las VMs utilizando herramientas como Metricbeat (recomendación) o agentes de monitoreo.

2. Elasticsearch:

Configurar un cluster de Elasticsearch con tres nodos.

- Cada nodo debe estar en una VM.
- Todos los nodos deben ser elegibles para ser master.
- Todos los nodos deben guardar datos.

Configuración de mappings y settings de los distintos índices.

- Creación de shards y copias ajustándose al cluster disponible.
- Optimización de índices para cada conjunto de datos. Mapping + settings
- Definición de los distintos campos (tipo + estructura), buscando optimizar el sistema.
- Crear analizadores para los datos ajustándose al uso posterior que se le va a dar.
- Utilización de plantillas de índices.

Queries a ELS.

- Para que los desarrolladores puedan acceder a los datos, desarrollo de scripts/notebooks para poder hacer queries a la base de datos de manera flexible.
- Utilizar API personalizadas para hacer queries a ELS.
- Personalizar queries con distintos parámetros que mejoren las búsquedas (ej: fuzziness).

Utilización de **Life Cycle Policies**.

- Utilización de políticas de vida automáticas para los índices que se considere (por ejemplo, índices de logs, noticias antiguas...)

Manejo de **API Keys** para Elasticsearch:

- Crear y gestionar API keys para controlar el acceso a Elasticsearch, definiendo permisos adecuados según los roles asignados (lectura, escritura, administración).
- Configurar políticas de acceso para cada API key, garantizando la seguridad y delimitación de las acciones permitidas.

- Acceso en logstash, scripts, notebooks... hecho a través de api keys y no con usuario y contraseña.

Prueba de redundancia de datos y cluster distribuido.

- Desarrollar una prueba donde se demuestre que el acceso a la información sigue siendo del 100% en el caso de que un nodo falle.
- Demostrar que es posible levantar el cluster desde cualquier nodo, si hay una caída de los servicios. y que es posible conectarse a Kibana desde cualquier nodo.

3. Kibana:

- Utilizar Kibana para crear visualizaciones que muestren:
 - Representación de datos que se consideren representativos del sistema.
 - Aspectos operativos: Uso de CPU y memoria, espacio en disco, ancho de banda... de las VMs donde estéis alojando los nodos de ELS.
 - **Configurar alertas en Kibana** sobre métricas clave de las VMs (uso de CPU, memoria, etc.) o de vuestros datos, proporcionando mecanismos de notificación automatizados para responder rápidamente a problemas operativos.

Programación de Inteligencia Artificial:

1. Api Rest

- Desarrollo de la API:
 - Diseñar endpoints RESTful claros y específicos para los recursos del sistema.
 - Implementar operaciones estándar de CRUD para los endpoints principales.
- Control de Versiones y Seguridad:
 - Crear un esquema de validación con JSON Schema para validar solicitudes y respuestas.
 - Implementar serialización y deserialización de datos en formato JSON.
- Control de Versiones y Seguridad:
 - Configurar autenticación y autorización en los endpoints.
- Monitorización y Logging:
 - Implementar un sistema de logging detallado.
- Documentación:
 - Generar documentación autogenerada con Swagger u otra herramienta compatible.

2. Scraper

- Extracción y Configuración:
 - Implementar módulos para extracción de datos web.
 - Configurar tareas periódicas (cron jobs) para actualizaciones regulares.
- Formato y Almacenamiento:
 - Transformar y almacenar los datos en formato JSON o CSV.
 - Diseñar la estructura de datos configurable según las necesidades del origen.
- Capa de Almacenamiento:
 - Integrar una base de datos SQL o NoSQL para almacenar los datos extraídos.
 - Optimizar el rendimiento de consultas mediante índices.
- Gestión de Errores y Frecuencia:
 - Implementar un sistema de reintentos y manejo de errores.
 - Configurar rate limiting y rotación de proxies para evitar bloqueos.
- Monitorización y Logging:
 - Registrar logs detallados con estado de solicitudes, tiempo de procesamiento y errores encontrados.
- Despliegue:

- Desplegar el scraper en un contenedor (Docker).
- Decidir qué partes necesitan automatización parcial o integral.

3. LLM Manager

- Interfaz de usuario:
 - Crear una interfaz con Gradio para interacción directa con el LLM.
- Modos de Ejecución:
 - Configurar un servidor para ejecutar el modelo LLaMA usando Hugging Face.
 - Implementar una segunda configuración para enviar solicitudes a la API de OpenRouter.
- Gestión de Modos:
 - Desarrollar un sistema para conmutar entre el modo LLaMA y el modo OpenRouter (archivos de configuración y variables de entorno)
- Monitorización y Logging:
 - Registrar interacciones con el modelo
 - Consultas enviadas.
 - Respuestas generadas.
 - Tiempo de procesamiento.
 - Errores.
- Escalabilidad y Recursos:
 - Configurar optimización de recursos para CPU y GPU en el servidor LLaMA.
 - Implementar rate limiting para la integración con OpenRouter.

4. Gitlab

- Control de Versiones y Organización del Proyecto
 - Definir una estructura clara de ramas para cada funcionalidad
 - Escribir **commits descriptivos y detallados** que expliquen claramente los cambios realizados en el código.
 - Mantener una convención de nombres para ramas y commits.
 - Establecer una política de revisiones de código.
- Integración continua y desarrollo continuo
 - Configurar pipelines funcionales en GitLab .
 - Asegurar que el pipeline cubra todos los componentes del proyecto.

Todos los productos desarrollador deberán estar en un repositorio con su documentación pertinente

Modelos de Inteligencia Artificial:

1. Selección del modelo de lenguaje:

- Investigar y seleccionar modelos de lenguaje natural existentes que se adapten a las necesidades del chatbot.
- Evaluar las características y limitaciones de los modelos seleccionados.
- Elegir el mejor modelo.

2. Diseño de la interfaz de usuario:

- Crear un diseño de interfaz de usuario para interactuar con el chatbot.
- Definir la estructura y el flujo de la conversación.

3. Creación del system prompt:

- Diseñar y crear un system prompt que defina el contexto y la tarea que el chatbot debe realizar.
 - Ajustar y refinar el system prompt para mejorar la precisión y la coherencia de las respuestas del chatbot.
4. Configuración del modelo de lenguaje:
 - Configurar el modelo de lenguaje seleccionado para que utilice el system prompt creado.
 - Ajustar los parámetros del modelo para optimizar su rendimiento.
 5. Pruebas y depuración:
 - Realizar pruebas exhaustivas para garantizar que el chatbot funcione correctamente y responda de manera coherente.
 - Identificar y solucionar errores y mejorar las métricas del chatbot.
 6. Implementación y despliegue:
 - Implementar el chatbot en un entorno de producción.
 - Configurar el chatbot para que esté disponible para los usuarios.
 - Añadir licencias de uso y consideraciones legales y éticas.
 7. Optimización y ajuste:
 - Monitorear el rendimiento del chatbot y ajustar el system prompt y la configuración según sea necesario.
 - Realizar ajustes para mejorar la experiencia del usuario.

Entregables:

Big Data Aplicado

Configuración de Logstash:

- **Archivos de configuración de Logstash** (.conf) para los pipelines de ingesta (tanto históricos como en tiempo real), así como el archivo **pipelines.yml**.
- **Documentación** que explique cada parte de la configuración, incluyendo las transformaciones aplicadas (p. ej. grok) y las fuentes de datos. **Readme detallado** con toda la información y explicaciones necesarias.

Cluster de Elasticsearch:

- **Archivo de configuración del clúster** (elasticsearch.yml) de cada nodo, especificando los cambios que se han realizado para poder desplegar el cluster en distintas MVs. Cualquier otro archivo que se haya tenido que modificar para el funcionamiento correcto del sistema.
- **Capturas de pantalla o logs** que evidencien la correcta configuración y puesta en marcha de los tres nodos del clúster.
- **Configuración de mappings y settings** para los índices definidos, junto con un index template automático para el log de scrapping.
- **Explicación** de las políticas de vida seleccionadas.
- **Listado de API Keys generadas** con las descripciones de cada una y los permisos otorgados (formato json) para diferentes propósitos.
- **Documentación** que describa el proceso de acceso a los distintos índices, con el uso de cada uno de los scripts/notebooks para poder hacer queries a la base de datos de manera flexible (acceso desarrolladores).

- **Prueba en tiempo real** de redundancia que demuestren la continuidad del acceso a los datos incluso con la caída de un nodo, demostración de conexión a kibana desde cualquier nodo y levantamiento del sistema desde cualquier nodo.
- **Informe** que describa las pruebas realizadas para asegurar la redundancia y alta disponibilidad del clúster.
- **Capturas o registros** mostrando cómo el sistema se comporta ante la caída de un nodo y cómo se mantiene el acceso a los datos.

Visualización y Monitorización (Kibana):

- **Dashboards de Kibana** exportados (formato .json) que muestren las visualizaciones de los datos representativos y los aspectos operativos de las VMs (uso de CPU, memoria, etc.)
- **Archivo de configuración** de Kibana (kibana.yml)
- **Capturas de pantalla/video** de los dashboards en funcionamiento.
- **Alertas configuradas** (descripción y evidencia) para el monitoreo de recursos.

Monitorización de las MVs:

- **Scripts de automatización (o similar)** (e.g. Metricbeat) que recopilen información del estado de las VMs.
- **Documentación** sobre el proceso de configuración de estas herramientas, explicando cómo se recopilan y almacenan los datos operativos.

Documentación General del Proyecto:

- **Instrucciones de despliegue** que incluyan pasos detallados para replicar la configuración (logstash, elasticsearch, kibana).

Programación de inteligencia artificial

Producto 1: API RESTful o API REST (api_service)

1. **Estructura y Estándares RESTful:** La API debe seguir las mejores prácticas de REST, ofreciendo endpoints claros y específicos organizados en torno a los recursos del sistema (e.g., [/datos](#), [/usuarios](#), [/consultas](#)). Además de operaciones estándar de CRUD, la API debe incluir un endpoint específico que permita enviar consultas a un modelo de lenguaje grande (LLM, explicado más adelante) mediante una petición POST (e.g., [/chat](#)). Este endpoint debe recibir entradas en JSON y devolver respuestas generadas por el modelo en el mismo formato, facilitando el acceso a capacidades avanzadas de procesamiento de lenguaje natural.
2. **Formato de Datos y Serialización:** Las respuestas y solicitudes deben manejarse en formato **JSON**, facilitando la interoperabilidad y la integración con otros servicios. Es recomendable el uso de un esquema de validación, como JSON Schema, para garantizar la consistencia y validar la estructura de los datos enviados y recibidos.
3. **Control de Versiones (Versioning):** La API debe estar versionada (por ejemplo, [/api/v1](#)) para facilitar el mantenimiento y la evolución de funcionalidades. Esto permite cambios y mejoras sin afectar a los sistemas externos que utilizan versiones anteriores.
4. **Monitorización y Logging:** Cada endpoint debe registrar logs detallados para auditar el acceso y el comportamiento, incluyendo el código de respuesta, el tiempo de procesamiento y la IP solicitante. La monitorización en tiempo real es clave para identificar patrones de uso y detectar anomalías.
5. **Documentación Autogenerada:** La API debe incluir documentación autogenerada y accesible (por **Swagger**).

Producto 2: Scrapper (scraper)

1. **Extracción Automatizada de Datos Web:** El scraper debe contar con módulos que permitan la extracción de información desde múltiples fuentes web de manera automatizada. Cada módulo debe soportar configuraciones específicas de extracción, como selectores CSS o XPath, para adaptarse a la estructura de cada sitio web objetivo. Además, el sistema debe permitir la recolección periódica de datos, gestionando cron jobs o tareas programadas para actualizar la información regularmente.
2. **Formato y Transformación de Datos:** Los datos extraídos deben ser transformados y almacenados en un formato estandarizado (e.g., JSON o CSV) para facilitar su uso y análisis posterior. La estructura de los datos debe ser configurable para permitir mapeos personalizados según el sitio web de origen, garantizando consistencia en el formato y alineación con las necesidades del usuario.
3. **Almacenamiento y Organización de Datos:** El sistema debe integrar una capa de almacenamiento eficiente, ya sea en una base de datos SQL, NoSQL o archivos planos, dependiendo del volumen y la estructura de los datos. La base de datos debe permitir consultas rápidas y filtrado para facilitar el acceso a información específica. Opcionalmente, se pueden incluir índices que optimicen el rendimiento en grandes volúmenes de datos.
4. **Gestión de Errores y Reintentos:** El scraper debe contar con un sistema de gestión de errores y reintentos para manejar problemas comunes en la extracción, como fallos de conexión, cambios en la estructura de la página o bloqueos por parte del sitio web. Esto incluye la implementación de un mecanismo de detección de errores, con notificaciones automáticas y reintentos configurables para garantizar la completitud de los datos.
5. **Control de Frecuencia y Respeto a Políticas de Acceso:** El scraper debe estar configurado para realizar peticiones de manera controlada, respetando las políticas de acceso de cada sitio web (como los tiempos de espera entre solicitudes y las limitaciones de acceso) para evitar bloqueos. Se recomienda el uso de técnicas de *rate limiting* y rotación de proxies para minimizar el riesgo de interrupciones en la extracción de datos.
6. **Monitorización y Logging:** Cada proceso de scraping debe registrar logs detallados que incluyan el estado de las solicitudes, el tiempo de procesamiento, los errores encontrados y el resultado de la extracción. La monitorización en tiempo real es clave para identificar rápidamente posibles bloqueos o cambios en los sitios web objetivo, permitiendo ajustes proactivos.
7. **Despliegue Containerizado y Automatizado:** El despliegue del sistema debe estar containerizado justificando que parte necesitan (o no) automatización parcial o integral.

Producto 3: LLM manager (llm_manager)

Interfaz de Usuario con Gradio para Interacción Directa: El agente LLM debe permitir la interacción directa con el LLM (aunque no contara con los datos). Esta interfaz debe estar integrada en un servidor y será accesible via IP.

Soporte para dos Modos de Ejecución:

- **Modo Servidor con modelos de Hugging Face:** En su configuración principal, el agente debe estar alojado en un servidor que ejecute el modelo LLaMA. Este servidor debe ser capaz de procesar las solicitudes de la interfaz Gradio y devolver respuestas generadas por el modelo de manera eficiente, permitiendo la ejecución de inferencias en tiempo real y optimizando el uso de recursos para manejar múltiples peticiones concurrentes. <https://huggingface.co/>
- **Modo Pruebas con API de OpenRouter:** Para temas de testeo y comparación de resultados, el agente debe incluir una segunda configuración que envíe las consultas a la API de OpenRouter. Esta configuración permite realizar pruebas rápidas con modelos externos, facilitando la evaluación de

rendimiento y calidad de respuesta entre LLaMA y otros modelos disponibles a través de OpenRouter. La integración con OpenRouter debe ser segura y permitir la personalización de las solicitudes según los parámetros específicos del entorno de testeo. <https://openrouter.ai/>

Sistema de Gestión de Versiones: El agente debe permitir la fácil conmutación entre los dos modos (LLaMA y OpenRouter), garantizando que cada versión esté aislada y que los cambios en una no afecten a la otra. Esto puede lograrse mediante configuraciones gestionadas por entorno (e.g., archivos de configuración o variables de entorno) para activar el modo deseado sin necesidad de modificar el código principal.

Monitorización y Logging de Interacciones: Cada interacción con el modelo, ya sea en el modo LLaMA o mediante OpenRouter, debe ser registrada. Esto incluye los logs de las consultas enviadas, respuestas generadas, tiempo de procesamiento y cualquier error ocurrido durante la inferencia. Estos registros son cruciales para evaluar la precisión y eficiencia de cada configuración del agente.

Escalabilidad y Gestión de Recursos: El servidor que ejecuta el modelo LLaMA debe estar configurado para optimizar el uso de CPU y GPU, gestionando la concurrencia de solicitudes y permitiendo el escalado según el número de usuarios. Para OpenRouter, la integración debe incluir un sistema de *rate limiting* para evitar sobrecargas y cumplir con los límites de la API.

Modelos de IA

Un documento que explique el código del modelo de lenguaje + lo scripts de código:

- Selección del modelo de lenguaje: modelo seleccionado y comparación con el resto de modelos que se han investigado, justificar por qué se ha elegido ese modelo y no otro.
- Definir la estructura y el flujo de la conversación.
- Diseño y creación del system prompt, pruebas y ajustes para refinar el system prompt.
- Configuración del modelo de lenguaje y ajuste de los parámetros del modelo para optimizar su rendimiento.
- Pruebas y depuración: pruebas exhaustivas para garantizar que el chatbot funcione correctamente y responda de manera coherente. Identificación y solución de errores y mejora de la precisión del chatbot.
- Implementación y despliegue: implementación del chatbot en un entorno de producción, para que esté disponible para los usuarios.
- Optimización y ajuste: selección y prueba de hiperparámetros, cálculo de métricas.

Documento de principios éticos y legales:

- Privacidad de datos.
- Licencias de las herramientas, librerías... utilizadas.
- Licencia de uso del propio modelo.
- Protección frente a errores.
- Consideraciones legales y éticas.

2.Criterios de evaluación

La evaluación del reto se refleja en la matriz de evaluación de competencias del mismo.

Criterio	1- Insuficiente	2- Deficiente	3 - Aceptable	4 - Bueno	5 - Excelente
Big Data Aplicado					
Ingesta de Datos con Logstash (15%)	Archivos incompletos o con errores graves. No se utilizan plugins adecuadamente.	Configuración básica con errores menores. Uso limitado de plugins. No usa syncedb.	Configuración funcional. Uso correcto de grok y otros plugins, sin optimizar. Uso de syncedb.	Configuración bien estructurada, plugins correctamente utilizados. Output dinámico adecuado. Utiliza pipelines.	Configuración avanzada y eficiente. Uso óptimo de grok y otros plugins. Output dinámico verificado y con pipelines.
Automatización de Métricas Operativas (10%)	No implementado o insuficiente.	Parcialmente automatizado, datos limitados.	Implementación funcional, con margen de mejora. No se recoge info de todas las MV.	Automatización completa y bien documentada.	Implementación robusta y detallada, con validación completa.
Configuración del Clúster de Elasticsearch (15%)	Errores críticos en la asignación de roles o conectividad.	Configuración básica del clúster con problemas de eficiencia o redundancia.	Clúster funcional con tres nodos, roles definidos, faltan optimizaciones.	Clúster configurado según especificaciones, roles correctamente asignados.	Configuración óptima, roles especializados, alta disponibilidad asegurada.
Optimización y Configuración de Índices (15%)	Mappings y settings mal definidos, sin optimización.	Definición básica, mejora posible en la estructura.	Configuración adecuada con margen de mejora.	Definición eficiente, índices bien optimizados, usando plantillas.	Configuración óptima usando plantillas, analizadores personalizados, ajustes finos según el uso específico.

Queries a Elasticsearch (10%)	Falta de desarrollo de queries.	Queries desarrolladas en scripts básicos, funcionalidad limitada.	Scripts funcionales, consultas básicas, uso de la API.	Scripts completos y flexibles, consultas diversas y con ciertos parámetros.	Scripts documentados, consultas complejas y optimizadas.
Prueba de Redundancia y Disponibilidad (10%)	No se ha implementado o la prueba.	Implementación incompleta, sin garantía de disponibilidad.	Prueba funcional, sin documentación clara.	Prueba implementada, evidencia de disponibilidad de datos. Cluster iniciable o kibana accesible desde cualquier nodo	Prueba exhaustiva y documentada, disponibilidad total garantizada. Cluster iniciable y kibana accesibles 100%.
Manejo de API Keys para Elasticsearch (5%)	Las API keys no se han generado o se han configurado incorrectamente, presentando graves problemas de seguridad.	Las API keys han sido generadas, pero la configuración de permisos es incorrecta o incompleta.	API keys generadas con permisos básicos, pero no todas están bien delimitadas o ajustadas a los roles adecuados.	API keys bien generadas, con permisos adecuados según los roles asignados y bien documentadas.	API keys generadas de manera óptima, con permisos precisos y estrictos según las necesidades de seguridad, además de estar bien documentadas con ejemplos claros de uso.
Utilización de Life Cycle Políticas (5%)	No se tiene en cuenta ninguna política de ciclo de vida.	Se ha generado una política de ciclo de vida, pero no se aplica bien a los índices.	Se utiliza una política de vida básica correctamente, pero sin un objetivo claro.	Se utiliza una política de vida correcta, con sentido.	Se optimiza el uso de políticas de vida en distintos casos, buscando optimizar el sistema.
Visualización y Monitorización en Kibana (10%)	No se han creado dashboards o son irrelevantes.	Visualizaciones básicas con datos incompletos.	Dashboards funcionales y representativos, sin configuraciones avanzadas.	Dashboards organizados, visualizaciones y métricas operativas clave.	Visualizaciones completas y optimizadas, con alertas para monitoreo proactivo.
Documentación y Presentación del Proyecto (5%)	Documentación incompleta o sin claridad.	Documentación básica, carente de detalles.	Documentación completa, explicaciones generales.	Documentación bien estructurada, explicaciones claras.	Documentación excelente, estructurada y detallada, evidencia de pruebas y resultados.

MIA	1- Insuficiente	2- Deficiente	3 - Aceptable	4 - Bueno	5 - Excelente
RA3 70%					
Selección del modelo de lenguaje 5%	No se investiga ni justifica la elección del modelo.	Se selecciona un modelo sin una comparación clara o justificación suficiente. Se omiten limitaciones.	Se ha investigado un número reducido de modelos y la justificación es general. Se consideran algunas limitaciones.	Se han investigado varios modelos y se justifica la selección con criterios adecuados, aunque falta una comparación detallada.	Se ha investigado y comparado múltiples modelos, justificando la elección con base en rendimiento, eficiencia y aplicabilidad. Se consideran limitaciones y ventajas.
Estructura y flujo de la conversación 5%	No se define una estructura para la conversación.	La estructura es confusa y carece de un flujo claro.	Se plantea una estructura básica sin profundidad en escenarios alternativos.	Se diseña una estructura comprensible, aunque con algunos puntos de mejora en la fluidez o manejo de errores.	Se define una estructura clara y lógica del flujo conversacional, con manejo de errores y diferentes escenarios bien planificados.
Diseño y ajuste del system prompt 5%	No se diseña un system prompt adecuado.	El prompt es poco efectivo y carece de ajustes significativos.	Se elabora un system prompt básico con mejoras limitadas.	Se diseña un system prompt funcional, con ajustes menores basados en pruebas.	Se crea un system prompt detallado y optimizado, refinado a través de pruebas iterativas para mejorar la coherencia y relevancia.
Configuración y ajuste del modelo 5%	No se realiza configuración alguna.	Se configura el modelo sin ajustes ni pruebas.	Se configuran parámetros sin ajustes detallados ni pruebas comparativas.	Se configuran parámetros básicos con ajustes razonables, aunque sin optimización completa.	Se optimizan hiperparámetros y se ajusta la configuración para maximizar rendimiento. Se documentan decisiones técnicas.
Pruebas y depuración 10%	No se realizan pruebas.	Se realizan pocas pruebas y se documentan errores sin corregir	Se llevan a cabo pruebas limitadas y se corrigen errores menores.	Se realizan pruebas con algunos escenarios clave, con correcciones	Se realizan pruebas exhaustivas con diferentes usuarios y

		adecuadamente.		aplicadas.	escenarios. Se documentan errores y se aplican mejoras.
Implementación y despliegue 10%	No se implementa el chatbot.	Se intenta implementar, pero con fallos funcionales.	Se implementa parcialmente o en un entorno de prueba sin acceso completo.	Se implementa, pero con algunas limitaciones técnicas o de accesibilidad.	Se implementa correctamente en un entorno de producción accesible para los usuarios.
Optimización y ajuste 10%	No se realizan optimizaciones.	Se identifican problemas, pero no se realizan mejoras significativas.	Se realizan pocos ajustes y sin evaluaciones rigurosas.	Se realizan algunos ajustes con base en pruebas, aunque sin un monitoreo continuo.	Se monitorea el rendimiento del chatbot, se ajustan hiperparámetros y se mejoran respuestas con base en métricas objetivas.
Relación con el procesamiento del lenguaje natural 5%	No se menciona el PLN.	Se hace una referencia muy superficial al PLN.	Se menciona el PLN sin profundidad en sus aplicaciones o limitaciones.	Se describe el PLN y su aplicabilidad, aunque con algunas lagunas conceptuales.	Se explica detalladamente el PLN, su potencial y limitaciones, vinculándolo con el chatbot desarrollado.
Papel del lingüista en IA 5%	No se menciona el papel del lingüista.	Se menciona de manera superficial y sin conexión clara.	Se menciona el papel del lingüista de forma general, sin aplicación específica al chatbot.	Se menciona la relevancia del lingüista, aunque sin detalles en la colaboración con informáticos.	Se justifica claramente el rol del lingüista en el desarrollo del chatbot, considerando colaboración con informáticos.
Desarrollo del sistema de PLN 10%	No se desarrolla un chatbot funcional.	Se desarrolla un chatbot con fallas en su funcionamiento.	Se crea un chatbot básico con funcionalidades limitadas.	Se desarrolla un chatbot con algunas limitaciones en fluidez o precisión.	Se desarrolla un chatbot funcional y bien optimizado para la tarea.
RA6 30%					
Riesgos legales y éticos 5%	No se mencionan riesgos legales ni éticos.	Se listan riesgos de forma superficial sin justificación ni soluciones.	Se mencionan algunos riesgos legales y éticos sin un análisis profundo ni ejemplos.	Se identifican los riesgos más relevantes con justificación adecuada, aunque sin ejemplos detallados.	Se analizan y argumentan detalladamente los riesgos legales y éticos de la IA, proporcionando ejemplos y

					soluciones.
Privacidad de datos 5%	No se aborda la privacidad de datos.	Se menciona de manera superficial sin claridad en la aplicación.	Se menciona la privacidad, pero sin conectar con normativas ni explicar cómo se garantiza.	Se menciona la privacidad de datos con referencias a normativas, aunque sin profundizar en su aplicación.	Se reconoce y explica con profundidad la importancia de la privacidad de datos, cumpliendo con normativas (RGPD, CCPA, etc.).
Cumplimiento legal 5%	No se menciona el cumplimiento legal.	Se menciona el cumplimiento legal sin especificar normativas.	Se hace referencia a regulaciones de manera genérica sin detalles en su aplicación.	Se mencionan normativas relevantes, aunque sin profundizar en su aplicación específica.	Se expone detalladamente el marco legal aplicable, incluyendo regulaciones internacionales y locales.
Protección frente a errores y ataques 2,5%	No se considera la protección ante errores o ataques.	Se mencionan riesgos sin proponer estrategias claras de mitigación.	Se incluyen medidas de seguridad básicas sin evaluación de riesgos detallada.	Se mencionan medidas de seguridad adecuadas, aunque sin profundizar en detalles técnicos.	Se describe un plan de seguridad robusto con estrategias de mitigación de errores y ataques.
Cumplimiento de normas éticas y legales (privacy by design) 5%	No se menciona la privacidad por diseño.	Se reconoce la necesidad de privacidad, pero sin integrar "privacy by design".	Se menciona "privacy by design" sin detalles sobre su implementación.	Se incluyen consideraciones de "privacy by design", pero sin aplicación detallada en todas las etapas.	Se demuestra un enfoque integral de "privacy by design", asegurando el cumplimiento de normativas en todas las etapas del desarrollo.
Licencias de uso 5%	No se mencionan las licencias de uso.	Se listan herramientas sin especificar sus licencias.	Se indican algunas licencias, pero sin relación clara con el proyecto.	Se mencionan las licencias utilizadas, aunque sin una explicación detallada.	Se describen y justifican detalladamente las licencias de uso de todas las herramientas, librerías y del modelo.
Corrección de sesgos de género y equidad 2,5%	No se consideran los sesgos de género ni la equidad en IA.	Se identifican sesgos sin ofrecer soluciones.	Se mencionan los sesgos, pero sin acciones concretas para corregirlos.	Se reconocen los sesgos y se proponen algunas estrategias de mitigación.	Se identifican y corrigen posibles sesgos en el modelo y sus aplicaciones, proponiendo mejoras.

Programación de IA					
Componente	1 - Insuficiente	2 - Deficiente	3 - Aceptable	4 - Bueno	5 - Excelente
Scraper (30%)	Estructura y Configuración (10%): No permite la extracción de los datos que se necesitan	Configuración básica con adaptabilidad limitada a algunas fuentes web	Configuración funcional con soporte para algunas personalizaciones básicas	Configuración flexible, adaptándose bien a varias fuentes web	Configuración avanzada y totalmente personalizable para múltiples fuentes web
	Extracción y Transformación de Datos (10%): Los datos extraídos no son consistentes ni están en un formato estándar	Los datos tienen problemas de consistencia y no siguen un formato definido	Datos extraídos en un formato estándar, aunque con algunos problemas de consistencia	Extracción y transformación consistentes, con formato estándar (JSON/CSV)	Extracción y transformación completas, consistentes y altamente configurables para cada fuente
	Gestión de Errores y Logs(10%): Sin manejo de errores ni control de frecuencia, bloqueos frecuentes	Manejo básico de errores, pero no respeta políticas de acceso	Gestión de errores y control de frecuencia básicos, con algunas limitaciones	Manejo robusto de errores y reintentos, respetando políticas de acceso	Sistema completo de manejo de errores y políticas de acceso, incluyendo rate limiting y proxies
API (30%)	Interacción RESTful y Estructura (10%): La API no sigue prácticas RESTful y los endpoints están desorganizados	La API tiene una estructura básica, pero con endpoints confusos	Sigue prácticas RESTful en su mayoría, aunque algunos endpoints podrían mejorar	Estructura bien definida y práctica, con endpoints claros y organizados	Diseño RESTful completo, con endpoints bien estructurados y documentados
	Procesamiento de Solicitudes y Respuestas (10%): Procesamiento de solicitudes y	Procesamiento básico, con JSON sin validación formal	Procesamiento en JSON funcional, con validación limitada	Procesamiento consistente en JSON, con esquema de validación y estructura clara	Procesamiento avanzado en JSON, validación formal con JSON Schema

	respuestas inconsistente, sin formato JSON estándar				
	Documentación y Logging (10%): No cuenta con documentación ni registros claros	Documentación mínima y logs parciales de interacción	Documentación y logs básicos, suficientes para la mayoría de interacciones	Documentación detallada y logs completos, accesibles y útiles	Documentación completa (Swagger/Postman), con logs detallados y bien estructurados
LLM Agent (20%)	Interfaz y Modo de Ejecución (10%): La interfaz es confusa, sin interacción directa con LLaMA ni conmutación de modo	La interfaz permite una interacción básica, pero la conmutación entre modos es complicada	Interfaz funcional para LLaMA, aunque con limitaciones en la conmutación	Interfaz intuitiva y flexible, con alternancia entre LLaMA y OpenRouter	Interfaz completamente funcional y eficiente, permite conmutación rápida y sin problemas
	Monitorización y Gestión de Recursos (10%): Sin monitoreo ni optimización de recursos	Monitoreo limitado y gestión de recursos básica	Monitoreo funcional, con gestión básica de recursos para concurrencia	Monitoreo detallado y gestión de recursos optimizada para solicitudes concurrentes	Monitorización completa y optimización avanzada de CPU/GPU, con concurrencia eficiente
GitLab (20%)	Control de Versiones y Organización del Proyecto (10%): Repositorio desorganizado, sin commits descriptivos ni ramas	Organización básica con algunos commits descriptivos, pero sin ramas	Estructura funcional, con commits descriptivos y ramas para algunas funcionalidades	Repositorio bien organizado, con commits detallados y ramas bien definidas	Control de versiones completo, con estructura clara, commits descriptivos y ramas para cada funcionalidad
	Uso de CI/CD (10%): No hay CI/CD configurado o pipelines no funcionales	CI/CD configurado parcialmente, pero con errores en los pipelines	CI/CD funcional, aunque no cubre todos los componentes del proyecto	CI/CD configurado, con pipelines funcionales y pruebas automatizadas	CI/CD avanzado, con pipelines completos, pruebas automáticas y despliegue continuo

Aspecto Evaluado	1 (Muy Insuficiente)	2 (Insuficiente)	3 (Suficiente)	4 (Bueno)	5 (Excelente)
Colaboración en GitLab	No se observa actividad significativa en el repositorio o contribuciones individuales.	Poca actividad o contribuciones irregulares, con escasa interacción entre miembros.	Colaboración adecuada con contribuciones individuales regulares, pero sin optimizar la organización.	Buen trabajo colaborativo con contribuciones claras, uso correcto de ramas y commits descriptivos.	Excelente colaboración, contribuciones constantes y bien organizadas. Uso adecuado de Pull Requests y resolución de conflictos.
Gestión de Tareas en Jira	No se han utilizado adecuadamente las tareas o sprints en Jira.	Tareas creadas pero no asignadas ni gestionadas correctamente.	Tareas y sprints definidos y parcialmente gestionados. Participación limitada en la planificación.	Uso adecuado de Jira, con tareas claramente asignadas y sprints bien gestionados.	Excelente uso de Jira. Las tareas están bien definidas, asignadas, y se ha hecho seguimiento constante. Proactividad en la planificación y actualización.
Trabajo Colaborativo y Comunicación	No hay evidencia de comunicación efectiva ni trabajo colaborativo.	Comunicación limitada o problemas frecuentes para trabajar juntos.	Comunicación adecuada y colaboración puntual. Participación limitada en las reuniones.	Buena comunicación, participación activa en reuniones y resolución de problemas colaborativamente.	Comunicación efectiva y constante, excelente organización del equipo. Participación activa en todas las reuniones y proactividad para resolver problemas.

Como procedimiento general de evaluación del reto:

- o Para dar un reto por aprobado en un módulo, la nota del reto deberá ser ≥ 5 (sobre 10) en ese módulo, teniendo en cuenta que hay que tener un mínimo de 2 (sobre 5) en todos los RAs de ese módulo, y que la nota de las prácticas, exámenes y pruebas de validación (si las hubiera) deberán ser ≥ 2 (sobre 5) para hacer media
- o En los retos intermodulares, puede suceder que un alumno/a apruebe el reto en un módulo y lo suspenda en otro; en este caso, realizaría la recuperación sólo del módulo que haya suspendido.
- o En función de los contenidos de cada módulo, el profesorado considerará realizar o no actividades de validación, como una actividad conceptual/procedimental donde el alumnado demuestre los conocimientos adquiridos en el reto; esta validación será tomada como una evidencia más a la hora de la calificación.
- Si todo el equipo suspende el reto (en 1 o varios módulos), el equipo completo mejorará la documentación y/o presentación del reto.
- Si sólo parte del equipo suspende el reto, como se terminan los días lectivos, no va a ser posible que el equipo completo colabore para mejorar el reto. En este caso, el profesorado considerará la

realización de alguna(s) actividad(es) específica(s) en las que se demuestre, individualmente, los conocimientos adquiridos aplicados al reto; dichas actividades pueden realizarse en común entre varios módulos, o por separado, a criterio del profesorado, y pueden realizarse dentro de las actividades del reto, o con una prueba escrita o con una prueba práctica o con la mejora de la presentación.

- El alumno/a deberá recuperar la(s) parte(s) que tenga suspendida(s).

Se realiza el feedback con cada equipo, donde se reflejen y se comenten, por una parte, los aspectos en los que el equipo ha funcionado bien, y por otra, se pueda como preguntar qué se podría haber hecho para hacerlo mejor.

NOTA: Esta temporalización se repetiría cada semana.